

API de Agendamento de Consultas

Relatorio tecnico final e rastreabilidade do capstone

Autora	Nathalia Artigas
Assessment	DR2 - Seguranca de Software
Resultado	34 testes; 94,64% de cobertura; ZAP sem alerta alto, medio ou baixo
Decisao	Aprovado para avaliacao local; producao bloqueada ate hardening operacional

Escopo

Arquitetura, decisoes dos exercicios 1 a 13, rastreabilidade de ameacas, auditoria OpenAPI, OWASP ZAP antes/depois, testes, riscos residuais e decisao de deploy.

1. Sumário executivo

A aplicação foi concluída como uma API FastAPI modular para consultas médicas, com três perfis humanos, integração M2M, persistência SQLAlchemy, autenticação OAuth2/JWT, autorização por papel e por recurso, validação fechada, hardening HTTP e pipeline DevSecOps. O capstone adicionou testes unitários com mocking, auditoria automática do OpenAPI e duas execuções passivas do OWASP ZAP.

A primeira execução do ZAP encontrou dois alertas médios, cinco baixos e dois informativos, todos concentrados na interface de desenvolvimento Swagger UI. Foram aplicados CSP completa, Subresource Integrity, dependências externas versionadas, políticas cross-origin, Permissions Policy e prevenção de cache. A segunda execução não registrou alerta médio, alto ou baixo; restaram somente duas observações informativas. O security gate foi aprovado.

O resultado é adequado para demonstração acadêmica e desenvolvimento local. O deploy em produção continua **bloqueado** até existirem evidências de TLS na borda, banco gerenciado e criptografado, segredos em cofre, rate limiting distribuído, MFA real, auditoria append-only, monitoramento e backup testado. Essa decisão decorre do domínio de saúde e da ausência atual de infraestrutura de produção, não de um finding de código ainda aberto.

2. Arquitetura e propriedades de segurança

O fluxo principal é: cliente HTTP -> middlewares -> autenticação -> autorização -> router -> serviço -> SQLAlchemy -> banco. A recepção recebe HTML renderizado com autoescape; o laboratório usa um token Client Credentials separado e só consulta disponibilidade agregada.

Propriedade	Implementação	Evidência principal
Exposição mínima	response_model e schemas de saída	app/routes/appointments.py, app/schemas/appointment.py
Entrada fechada	Pydantic, regex, limites e extra="forbid"	app/schemas/appointment.py
Autenticação	OAuth2PasswordBearer, bcrypt e JWT expirável	app/security/authentication.py, app/security/jwt.py
Autorização	RBAC + ownership + scopes	app/security/authorization.py
Separação M2M	Client Credentials, token_type e escopo dedicado	app/routes/auth.py, app/routes/availability.py
Persistência	SQLAlchemy e consultas parametrizadas	app/services/appointments.py
Segurança no browser	autoescape, CSP, SRI e headers	app/routes/reception.py, app/security/middleware.py, app/routes/docs.py
Proteção contra abuso	rate limits diferenciados	app/security/middleware.py
Verificação contínua	pytest, Ruff, Bandit, Trivy, ZAP e gate	.github/workflows/security.yml

3. Decisões por exercício

Exercício	Decisão e resultado	Evidência
1	Ambiente virtual, módulos separados e CRUD REST por APIRouter; primeiro teste de sucesso.	app/, tests/test_appointments.py
2	Schemas distintos de entrada/saída e Jinja2 com herança e autoescape. Campos internos não entram na resposta.	app/schemas/appointment.py, app/templates/
3	CIA aplicada com confidencialidade elevada por dados de saúde; DFD registra fronteiras e fluxos.	docs/cia_frameworks.md, docs/dfd.md
4	Misuse cases e STRIDE geraram TM-01 a TM-10, com ativos, atacantes, superfícies e mitigações.	docs/misuse_cases.md, docs/matriz_stride.md, docs/threat_model.md
5	Arquitetura dividida em borda, API, autenticação, domínio, persistência e apresentação; design, implementação e infraestrutura tratados separadamente.	docs/arquitetura_seguranca.md
6	OAuth2PasswordBearer, bcrypt, JWT curto e MFA simulado. RBAC é combinado com ownership porque papel sozinho não impede BOLA.	docs/autenticacao_autorizacao.md, tests/test_authorization.py
7	Client Credentials para o laboratório, com identidade de serviço e escopo availability:read, sem acesso ao CRUD clínico.	tests/test_m2m.py
8	Revisão manual identificou BOLA, abuso de autenticação, revogação tardia, configuração insegura, mass assignment, XSS e risco de SQL injection.	docs/owasp_vulnerabilidades.md
9	Ownership central, entrada por allowlist, extra="forbid", autoescape e SQL parametrizado. A correção foi propagada a rotas com a mesma causa.	tests/test_security_regressions.py, tests/test_templates.py
10	CORS explícito, HSTS, anti-frame, nosniff, CSP e limites distintos para login humano e M2M.	docs/hardening_http.md, tests/test_http_security.py
11	SQLModel, sessão por dependência e configuração BaseSettings; somente .env.example é versionado.	app/database/, app/config.py, .env.example
12	CI em jobs independentes para testes, SAST, SCA e DAST; gate agregado obrigatório na branch principal.	docs/devsecops_pipeline.md, .github/workflows/security.yml
13	Mocking de entrada/autorização, auditoria OpenAPI 6/6, ZAP antes/depois e relatório final rastreável.	tests/test_mocked_security.py, scripts/audit_openapi.py, docs/evidencias/

4. Rastreabilidade: threat model, OWASP, controle e teste

Ameaça	Categoria OWASP	Controle/correção	Evidência automatizada	Estado
TM-01 - BOLA	API1:2023 Broken Object Level Authorization	Ownership central em leitura, atualização e exclusão; listagem filtrada.	tests/test_authorization.py, tests/test_security_regressions.py	Mitigada
TM-02 - CRUD anônimo	API2:2023 Broken Authentication	Bearer JWT e scopes em todas as operações clínicas.	tests/test_http_security.py, auditoria OAS-03	Mitigada
TM-03 - agenda nominal excessiva	API3:2023 Broken Object Property Level Authorization	Papéis permitidos e filtro do profissional.	tests/test_templates.py	Mitigada localmente; falta isolamento por clínica
TM-04 - confusão M2M/humano	API5:2023 Broken Function Level Authorization	token_type, scopes e dependências distintas.	tests/test_m2m.py, tests/test_authorization.py	Mitigada
TM-05 - brute force/DoS	API4:2023 Unrestricted Resource Consumption	Rate limiting diferenciado e Retry-After.	tests/test_authentication.py	Mitigada em uma instância; residual distribuído
TM-06 - repúdio/adulteração	A09:2021 Security Logging and Monitoring Failures	Principal e timestamps internos; campos não expostos.	testes de response model	Parcial; falta log append-only
TM-07 - cópia do banco	A02:2021 Cryptographic Failures	Configuração externa e acesso local mínimo.	inspeção de configuração	Residual; falta banco gerenciado/criptografado
TM-08 - stored XSS	A03:2021 Injection	Regex/allowlist, autoescape e CSP.	tests/test_templates.py, tests/test_appointments.py	Mitigada
TM-09 - mass assignment	API3:2023	extra="forbid" e saída por allowlist.	tests/test_mocked_security.py, auditoria OAS-06	Mitigada
TM-10 - agenda inconsistente	API4:2023	timezone obrigatório, validação e transação.	tests/test_security_regressions.py	Mitigada no processo; concorrência distribuída requer constraint

5. Auditoria OpenAPI

O script `python -m scripts.audit_openapi` lê o contrato gerado pela própria aplicação, evitando divergência de um arquivo mantido manualmente. O pipeline final aprovou os seis controles:

- `tokenUrl` OAuth2 aponta para `/api/v1/auth/token`;
- escopos humanos, administrativos e M2M estão declarados;
- nove operações protegidas documentam 401 e 403;
- todas as operações públicas documentam 429;
- nenhum campo interno sensível aparece nos schemas públicos;
- criação e atualização rejeitam propriedades extras.

O OpenAPI não expressa ownership por objeto; por isso BOLA continua coberta por testes HTTP e não é considerada provada apenas pelo contrato.

6. OWASP ZAP: execução inicial

Execução: 35468605011, ZAP Baseline passivo contra `http://127.0.0.1:8000/docs`. O relatório original está em `docs/evidencias/zap_before_hardening/`.

Plugin	Finding	Risco ZAP	Categoria	Correção aplicada
10055	CSP sem form-action	Médio	API8:2023 / A05:2021 Security Misconfiguration	CSP passou a declarar form-action, base-uri, object-src, fontes de script, estilo e imagem.
90003	Subresource Integrity ausente	Médio	A08:2021 Software and Data Integrity Failures	Swagger customizado usa versão exata e hashes SHA-384 em JS e CSS.

Plugin	Finding	Risco ZAP	Categoria	Correção aplicada
10017	JavaScript de domínio externo	Baixo	A08:2021 / API8:2023	Origem foi restrita a jsDelivr, versão fixada, SRI e crossorigin="anonymous"; risco de CDN reduzido.
90004	COEP ausente	Baixo	API8:2023	Cross-Origin-Embedder-Policy: require-corp.
90004	COOP ausente	Baixo	API8:2023	Cross-Origin-Opener-Policy: same-origin.
90004	CORP ausente	Baixo	API8:2023	Cross-Origin-Resource-Policy: same-origin.
10063	Permissions Policy ausente	Baixo	API8:2023	Câmera, microfone, geolocalização, pagamento e USB desabilitados.
10109	Aplicação web moderna	Informativo	Observação, não vulnerabilidade	Nenhuma ação necessária.
10049	Conteúdo armazenável/cacheável	Informativo	API8:2023	Cache-Control: no-store, max-age=0 e Pragma: no-cache.

7. OWASP ZAP: verificação após correção

Execução: 35517319082. Resultado: **zero alertas altos, médios ou baixos**. Permaneceram apenas:

Plugin	Observação	Interpretação
10109	Modern Web Application	Classificação informativa da Swagger UI; não requer correção.
10049	Non-Storable Content	Confirma que as respostas agora impedem armazenamento; é evidência do controle, não uma falha.

Os relatórios JSON, HTML e Markdown finais estão em docs/evidencias/zap/. O gate DAST aprovou a execução porque não havia `riskcode >= 3`; além disso, a revisão humana confirmou que não restou alerta médio ou baixo.

8. Testes, cobertura e pipeline

Validação local final:

- Ruff: aprovado;
- pytest: 34 testes aprovados;
- cobertura: 94,64%, acima do mínimo de 90%;
- auditoria OpenAPI: 6/6 controles aprovados.

O pipeline executa testes/cobertura, Bandit, Trivy e ZAP em paralelo. O job Security gate depende dos quatro resultados e bloqueia o merge se qualquer um falhar. Bandit bloqueia achados médios ou altos com confiança média/alta; Trivy bloqueia vulnerabilidades altas/críticas corrigíveis; ZAP bloqueia risco alto. O impacto de negócio eleva qualquer acesso indevido a dado de saúde a bloqueante, independentemente do score isolado.

Os testes com mocking confirmam duas propriedades que uma simples asserção de status não provaria: payload inválido não alcança a camada de serviço e autorização negada impede a atualização. Um terceiro teste registra a ordem autorização -> mutação.

9. Riscos residuais e decisão de deploy

Risco residual	Consequência	Tratamento necessário antes de produção
SQLite local sem evidência de criptografia	Cópia ou adulteração de dados em repouso	Banco relacional gerenciado, criptografia, rede privada e least privilege.
Rate limiter em memória	Contorno por múltiplas réplicas/IPs	Redis/gateway distribuído, limites por identidade e telemetria.
MFA simulado	Não comprova posse de segundo fator real	TOTP/WebAuthn, recuperação segura e política de enrollment.
Chave JWT simétrica sem rotação automatizada	Amplia impacto de segredo comprometido	Secret manager, rotação, kid e preferência por chaves assimétricas.
Auditoria não append-only	Administrador/host pode apagar rastros	Trilha imutável externa, correlação e alertas.
TLS/HSTS sem infraestrutura comprovada	Tráfego pode não estar protegido na borda	Proxy TLS, certificado, redirect e teste de configuração.
Backup/RPO/RTO não demonstrados	Perda de agenda ou recuperação lenta	Backup criptografado, restauração testada e responsáveis definidos.
Isolamento por clínica ainda não modelado	Acesso lateral entre unidades futuras	Introduzir clinic_id, política por tenant e testes cruzados.

Decisão: liberar para avaliação acadêmica e execução local controlada; **não autorizar deploy em produção**. O risco residual não é aceitável para dados de saúde sem os controles operacionais e de infraestrutura listados. O código não apresenta finding ZAP bloqueante, mas segurança de produção depende de evidências que este repositório, sozinho, não pode fornecer.

10. Integridade e reprodução

Os artefatos do pipeline têm digests publicados pelo GitHub e os relatórios foram copiados sem alteração para docs/evidencias/. O projeto pode ser reproduzido com:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-dev.txt
cp .env.example .env
ruff check .
pytest --cov=app --cov-fail-under=90 -q
python -m scripts.audit_openapi --output reports/openapi-audit.json
```

Nenhuma credencial real, .env, banco local, cache ou ambiente virtual deve integrar o ZIP final.

11. Conferência da rubrica

Os 24 critérios da rubrica foram revistos individualmente. A matriz docs/matriz_rubrica.md aponta, para cada item, a implementação, a evidência objetiva e a forma de reprodução. O resultado da auditoria é 24/24 critérios demonstrados. O vídeo não listado continua sendo a única evidência externa que depende da autora e deve ter sua URL registrada em VIDEO_LINK.txt antes da geração do ZIP definitivo.